

Chapter Four

Just Because You Can Does Not Mean You Should

A customer asked about hardship terms during a renewal conversation, and the company's own assistant told them what the policy was. The answer was fluent and specific. It described a standard hardship clause in exactly the register you would expect from a commercial policy document, using language drawn from previous negotiations and internal contract discussions. There was one problem: the company had no such policy.

What the assistant had done was combine old negotiation material, historical email language, and unapproved internal drafts into an answer that read like an official commitment. The customer, reasonably, treated it as one. Which left the com-

mercial team explaining to a customer, mid-renewal, why the company's own system had described a policy the company did not intend to honour.

The failure does not require anyone to have intended harm. That is the part worth sitting with.

This practitioner example is an illustrative composite, constructed to show a recurring organisational pattern rather than describe a single real company or incident. It was an AI service desk assistant at a global industrial group, built to reduce support load: it answered customer queries, handled internal IT and HR requests, retrieved from internal knowledge bases, opened tickets, and helped agents move routine interactions faster. Not an exotic use case: McKinsey's 2025 survey found agent use most commonly reported in IT and knowledge management, with service-desk management among the most commonly reported agentic applications.^[20]

It also worked. The first version came together quickly because the platform made the build feel unusually light. The model interpreted natural-language questions, retrieved relevant material, and produced answers good enough to use with minimal intervention. The internal demonstration created exactly the right kind of energy: shorter response times, lower queue pressure, visible relief for teams handling repetitive requests.

A limited pilot followed, and users found it helpful. Support managers watched queues shrink. Finance saw measurable capacity released. Leadership saw a proof point that the company

could move quickly with AI. Every one of those signals was real, and every one of them made pausing harder.

When legal, HR, security, and support leadership finally reviewed the system, it had already been running long enough to produce a substantial body of operational output. Transcript sampling showed the pattern you would expect and nobody wanted: most answers were ordinary and useful, and some were materially wrong. The assistant had fabricated refund policies, service-level commitments, contract interpretations, and HR entitlements in precisely the same fluent tone it used for everything it got right.

That last detail is the whole chapter. The system did not fail because it was useless. It failed because its usefulness could not be separated from its liability.

Capability is not permission, and permission without a named owner is not governance. The gap between a system that can be built and a system that should be allowed to run is where this chapter lives.

Accidental production

The assistant did not fail at the demonstration stage; it crossed from working prototype into operating dependency without anyone making a deliberate production decision.

This is accidental production, and it is worth distinguishing from shadow AI. Nothing here emerged outside official channels. The assistant began as an approved pilot and hardened

into an operational dependency simply because people relied on what it delivered. Utility did not merely precede the governance problem—it actively protected the system from being paused or removed.

The mechanics are unremarkable: the model handles natural language smoothly, the modern stack abstracts away operational friction, the demo succeeds, and the team keeps moving. At no point does the absence of governance look or feel like a breakdown—the system is visibly producing value, which is precisely what makes the trap so insidious.

This scenario illustrates how institutional incentives produce failure without requiring a reckless team. The squad was asked to demonstrate velocity, and the quarterly scorecard rewarded deployment speed, deflection rates, and cost reduction. Modern tooling made that acceleration easy. Meanwhile, no metric existed to track whether the deployment carried an accountable owner, a hardened data boundary, an operational baseline, or a validated fallback.

Predictably, usefulness became the default graduation criterion—not through a conscious decision, but through the absence of one. What goes unmeasured becomes optional the moment a deadline appears: there is no executive applause for a curated retrieval corpus or a tested circuit-breaker that never fires. A team evaluated on speed and business value will optimize for speed and business value; governability was simply never part of the contract.

The core enterprise problem is not velocity. It is demanding speed from teams without providing an operational path that makes governance the condition of moving fast.

Where it breaks

Four failures did the damage, and they recur across almost every version of this pattern.

- **Ownership.** The product squad owned the build. Support leadership owned the performance metric. IT owned parts of the platform. Security had reviewed some infrastructure assumptions. Legal and HR were accountable for the consequences of certain answers without operating the system that produced them. When the incident surfaced, the first serious question was not technical. It was managerial: who had the authority to pause, change, or retire this? That question should have had an answer before launch. A system without a named owner can still produce value. What it cannot produce is accountable control, and when behaviour drifts or a customer challenges an output, the organisation has to assemble authority after the fact. Emergency coordination wearing the name of governance.
- **The data boundary.** The assistant retrieved from internal knowledge bases that had accumulated for years: approved policy, obsolete policy, draft policy, nego-

tiation records, historical customer emails, old support macros, HR material, and documents that were technically accessible but never intended for broad operational use. Repository-level access control settled who was permitted to open a document. The assistant needed something else settled: which material it was allowed to use when responding to this class of question. The distinction matters more than it sounds. A human employee with access to a draft clause generally understands it is a draft. A retrieval system treats it as relevant context. Technically reachable is not institutionally valid, and that boundary cannot live in the prompt. It has to be designed into retrieval, indexing, approval, logging, and maintenance. Otherwise the system will eventually treat the archive as authority, which is exactly what happened.

- **Drift without detection.** The assistant did not become risky at an identifiable moment. It became risky gradually, as it met gaps in policy material, conflicting documents, and questions outside the pilot's narrow assumptions, and filled those gaps with plausible language. Some answers stayed useful. Others crossed into fabrication. Nothing in the operating process separated the two quickly enough. No systematic transcript review. No policy-violation sampling. No exception threshold. No owner comparing behaviour against approved boundaries. No defined signal that would trig-

ger intervention. The system was being judged on aggregate usefulness while risk accumulated inside individual interactions, which is how drift becomes expensive. It never announces itself as drift. It appears as a series of individually plausible answers, right up until one of them matters.

- **Scale before control.** Scaling transforms localized errors into systemic exposure. Both human agents and AI assistants can repeat a mistaken policy interpretation. An automated assistant can propagate that mistake across thousands of interactions before the organisation detects and corrects it. The value case and the risk exposure can grow together. Faster answers mean faster exposure when the answer is one the organisation cannot defend.

These four cluster because they are versions of one missing capability. The organisation could not say, reliably and at runtime, what the system was doing, what it was allowed to use, whether it was still behaving as intended, and who could change it. When those answers live only in project memory, they are not controls. They are assumptions.

Why a useful failure costs more

The assistant did real work. It answered routine questions, reduced queue pressure, and helped agents move faster, and that is precisely why the governance failure was so expensive.

A useless system is easy to remove. A useful ungoverned system forces a choice nobody wants to make: keep it running and accept unclear exposure, or take it offline and lose the productivity gain that justified the whole project.

Once legal, HR, security, and support leadership arrive after an incident, the default answer is to restrict. The organisation swings from optimism to blanket caution, and that overcorrection is part of the cost, though it never appears in the incident report.

One ungoverned deployment can make the next ten harder. Legal becomes less willing to approve. Security demands central review for systems that could have carried lighter controls. Business teams conclude that governance is a blocker, which by then is empirically true. Engineering becomes reluctant to experiment. The incident itself is recoverable in weeks. The loss of institutional trust around the AI programme can run for years.

This is why "move fast" is an incomplete instruction in enterprise AI. The question is not whether the team can move fast. It is whether the organisation can keep moving after the first serious failure. Governance is what makes speed recoverable.

The architectural absence

The diagnosis is structural. The system lacked a production path that made ownership, boundaries, monitoring, and fall-back non-optional.

Capability is not readiness. A system can be useful, impressive, and economically attractive and still be unfit for production, because the organisation cannot own how it behaves. Once AI moves from demonstration to dependency, governance stops being a future improvement and becomes part of the system, whether or not anyone has built it.

The service desk case exposed one ungoverned dependency, discovered the expensive way. The obvious next question is how many more are sitting in the estate right now, undiscovered, doing useful work.